Experiment 4: Best First Search

Aim:

To implement Best First Search algorithm that selects the next node based on a heuristic estimate of distance to the goal.

Algorithm:

1. Use a priority queue ordered by heuristic value h(n).
2. Insert (h(start), start, [start]).
3. While PQ is not empty:
   a. Pop the node with lowest heuristic.
   b. If it is the goal, return the path.
   c. Expand unvisited neighbors using their heuristic values.
4. Heuristic guides the search towards the goal.

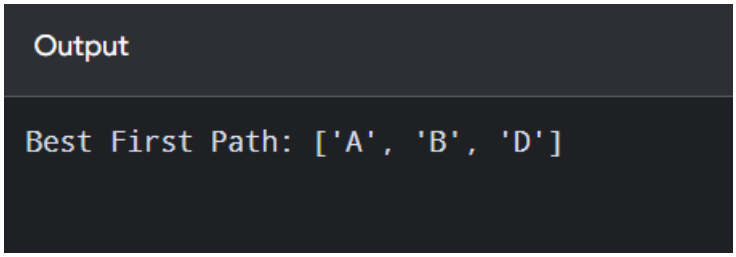
Code:

```
import heapq

def best_first(graph, start, goal, h):
    pq = [(h[start], start, [start])]
    visited = set()
    while pq:
        _, node, path = heapq.heappop(pq)
        if node in visited: continue
        visited.add(node)
        if node == goal:
            return path
        for n in graph.get(node, []):
            heapq.heappush(pq, (h[n], n, path + [n]))
    return None

graph = {'A':['B','C'], 'B':['D'], 'C':['D'], 'D':[]}
h = {'A':4, 'B':2, 'C':3, 'D':0}
print("Best First Path:", best_first(graph, 'A', 'D', h))
```

Sample Output:



Result:

Best First Search used the heuristic to preferentially expand node B (lower h value) and found the path ['A','B','D'].